P6 MQTT: Mosquitto, tópicos y autodescubrimiento desde Arduino y ESP32

Semana	16–20 nov 2026
Imparte	IES Federica Montseny
Duración	2 sesiones de 50 min + práctica en el aula
Material	Montaje Arduino + ESP32 de P3, HA del aula con el complemento Mosquitto, programa MQTT Explorer en el PC (opcional)
Requisitos previos	P3 y P4

Objetivos

- Entender el modelo publicar/suscribir y el papel del broker.
- Diseñar tópicos con la convención del proyecto.
- Publicar desde el ESP32 los datos que llegan del Arduino.
- Hacer que HA cree las entidades solo (MQTT discovery) y controlar un relé desde HA.

Contenidos

1. Cómo funciona MQTT

MQTT es un protocolo ligero de mensajería. Los dispositivos no se hablan entre sí: **publican** mensajes en un **tópico** y el **broker** los reparte a quien esté **suscrito**. En nuestra aula el broker es Mosquitto, que corre como complemento dentro de Home Assistant.

Concepto	Significado	En el proyecto
Broker	Servidor que recibe y reparte mensajes	Complemento Mosquitto, puerto 1883
Tópico	Ruta jerárquica separada por /	aula/vasco/clima/nodo2/temperatura
Comodines	+ un nivel, # todos los niveles	aula/+/clima/# (clima de todos los centros)
retain	El broker guarda el último mensaje	Mensajes de configuración y disponibilidad
QoS	0 como mucho una vez, 1 al menos una vez	Usaremos 0 para lecturas
LWT	Mensaje que el broker publica si el nodo se desconecta	«offline» en .../disponible

2. Convención de tópicos

`aula/<centro>/<reto>/<nodo>/<magnitud>` para las lecturas y `.../<actuador>/set` para las órdenes. Todo en minúsculas, sin tildes ni espacios.

3. Preparar el broker (profesorado o administrador de la célula)

- Ajustes → Complementos → Tienda → «Mosquitto broker» → Instalar → Iniciar (marcar «Iniciar al arrancar»).
- Crear un usuario de HA solo para los nodos (p. ej. *mqtt_nodos*), sin permisos de administrador: Mosquitto acepta usuarios de HA.
- Ajustes → Dispositivos y servicios: aparece la integración MQTT descubierta → Configurar.

Proyecto de agrupación de centros financiado por el Ministerio de Educación, Formación Profesional y Deportes.
El contenido de esta publicación compromete exclusivamente a sus autores.

4. Autodescubrimiento (MQTT discovery)

Si el nodo publica un mensaje JSON en `homeassistant/<tipo>/<id>/config` con retain, HA crea la entidad automáticamente con su nombre, unidad y dispositivo. Así no hay que escribir configuración en HA.

Sketch 6 · ESP32 pasarela: recibe del Arduino (P3) y publica en MQTT (librería PubSubClient de Nick O'Leary)

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* SSID = "RED_IOT_AULA";          const char* PASS = "********";
const char* BROKER = "192.168.50.10";       // IP del servidor de Home Assistant
const char* MQ_USER = "mqtt_nodos";         const char* MQ_PASS = "********";

const char* NODO    = "vasco_clima_nodo2";
const char* T_DISP  = "aula/vasco/clima/nodo2/disponible";
const char* T_TEMP  = "aula/vasco/clima/nodo2/temperatura";
const char* T_HUM   = "aula/vasco/clima/nodo2/humedad";
const char* T_VENT  = "aula/vasco/clima/nodo2/ventilador";
const char* T_VSET  = "aula/vasco/clima/nodo2/ventilador/set";

WiFiClient wifi;
PubSubClient mqtt(wifi);
float temperatura = NAN, humedad = NAN;

const char* DISP_JSON = "\"device\":{\"identifiers\":[\"vasco_clima_nodo2\"],"
                        "\"name\":\"Clima nodo 2\",\"model\":\"Arduino UNO + ESP32\"}";

void anunciarSensor(const char* id, const char* nombre, const char* topic,
                    const char* unidad, const char* clase) {
  char cfg[96], json[480];
  snprintf(cfg, sizeof(cfg), "homeassistant/sensor/%s/config", id);
  snprintf(json, sizeof(json),
    "{\"name\":\"%s\",\"unique_id\":\"%s\",\"state_topic\":\"%s\","
    "\"unit_of_measurement\":\"%s\",\"device_class\":\"%s\","
    "\"availability_topic\":\"%s\",%s}", nombre, id, topic, unidad, clase, T_DISP, DISP_JSON);
  mqtt.publish(cfg, json, true);
}
void anunciarInterruptor() {
  char json[480];
  snprintf(json, sizeof(json),
    "{\"name\":\"Ventilador\",\"unique_id\":\"vasco_clima_nodo2_vent\","
    "\"command_topic\":\"%s\",\"state_topic\":\"%s\","
    "\"availability_topic\":\"%s\",%s}", T_VSET, T_VENT, T_DISP, DISP_JSON);
  mqtt.publish("homeassistant/switch/vasco_clima_nodo2_vent/config", json, true);
}
void alRecibir(char* topic, byte* datos, unsigned int len) {   // órdenes desde HA
  String orden;
  for (unsigned int i = 0; i < len; i++) orden += (char)datos[i];
  bool on = (orden == "ON");
  Serial2.println(on ? "R:1" : "R:0");                  // se lo pasa al Arduino (relé)
  mqtt.publish(T_VENT, on ? "ON" : "OFF", true);        // confirma el estado
}
void conectarMQTT() {
  while (!mqtt.connected()) {
    // última voluntad (LWT): si el nodo cae, el broker publica "offline"
    if (mqtt.connect(NODO, MQ_USER, MQ_PASS, T_DISP, 0, true, "offline")) {
      mqtt.publish(T_DISP, "online", true);
      anunciarSensor("vasco_clima_nodo2_temp", "Temperatura", T_TEMP, "°C", "temperature");
      anunciarSensor("vasco_clima_nodo2_hum",  "Humedad",     T_HUM,  "%",  "humidity");
```

```
      anunciarInterruptor();
      mqtt.subscribe(T_VSET);
    } else {
      delay(3000);
    }
  }
}
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, 16, 17);   // enlace con el Arduino (P3)
  WiFi.begin(SSID, PASS);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  mqtt.setServer(BROKER, 1883);
  mqtt.setBufferSize(512);                   // los JSON de discovery superan los 256 B por defecto
  mqtt.setCallback(alRecibir);
}

void loop() {
  conectarMQTT();
  mqtt.loop();
  if (Serial2.available()) {
    String linea = Serial2.readStringUntil('\n');
    // procesar(linea) de P3 rellena temperatura y humedad
    procesar(linea);
    mqtt.publish(T_TEMP, String(temperatura, 1).c_str());
    mqtt.publish(T_HUM,  String(humedad, 1).c_str());
  }
}
```

Copia la función *procesar()* del Sketch 3b encima de *loop()*. En el Arduino añade la lectura de «R:1 / R:0» por el enlace serie para activar el relé (lo montaremos con seguridad en P9; por ahora, usa un LED).

Práctica simultánea en el aula

1. Comprueba con el profesor que Mosquitto está en marcha y que tienes usuario y contraseña de MQTT para tu célula.
2. Instala la librería PubSubClient en Arduino IDE.
3. Adapta el Sketch 6: códigos de tu centro, nombre de nodo de tu célula e IP del servidor. Carga el código en el ESP32.
4. En HA → Ajustes → Dispositivos y servicios → MQTT → Configurar, escucha el tópico *aula/#* y comprueba que llegan tus mensajes.
5. Localiza el dispositivo «Clima nodo 2» creado por autodescubrimiento, asígnalo al área y renombra los ID de entidad según la convención.
6. Actúa sobre el interruptor «Ventilador» desde HA y comprueba que el Arduino recibe «R:1» y enciende el LED.
7. Desenchufa el ESP32: en menos de un minuto las entidades deben pasar a «No disponible» gracias al LWT.

Lista de comprobación (evidencias)

- ☐ Mensajes visibles en el tópico aula/<centro>/... desde la herramienta de escucha de HA o MQTT Explorer.
- ☐ Dispositivo creado por discovery con temperatura, humedad e interruptor.
- ☐ Orden desde HA que llega al Arduino.
- ☐ Disponibilidad «offline» al desconectar el nodo.

Proyecto de agrupación de centros financiado por el Ministerio de Educación, Formación Profesional y Deportes.
El contenido de esta publicación compromete exclusivamente a sus autores.

Errores frecuentes

Síntoma	Solución
rc=5 al conectar (no autorizado)	Usuario o contraseña de MQTT incorrectos, o el usuario no existe en HA.
Las lecturas llegan pero no se crea la entidad	El JSON supera el búfer: falta setBufferSize(512), o hay un error de comillas en el JSON.
Entidades duplicadas	Cada entidad necesita un unique_id distinto y estable; no lo cambies entre cargas.
La orden llega pero el estado no cambia en HA	Hay que publicar el estado en el state_topic tras ejecutar la orden.

Para ir más allá (nivel avanzado)

- Envía temperatura y humedad en un único JSON en .../estado y usa value_template en el discovery.
- Suscríbete a aula/+/clima/+/temperatura y compara en MQTT Explorer las temperaturas de otras células.

Conexión con FpAR y autoevaluación

> **Competencia transversal: Resiliencia, flexibilidad y agilidad**: la conexión MQTT falla a menudo al principio; la célula registra en su diario cada error y cómo lo superó.

Tres preguntas para comprobar lo aprendido:

1. ¿Qué diferencia hay entre los comodines + y #?
2. ¿Para qué sirve publicar los mensajes de configuración con retain?
3. ¿Qué publica el broker cuando un nodo con LWT se desconecta sin avisar?

Proyecto de agrupación de centros financiado por el Ministerio de Educación, Formación Profesional y Deportes.
El contenido de esta publicación compromete exclusivamente a sus autores.